LaRT v2.10 User Manual

Contents

1. Introduction **3**

2. Installation and Build **3**

2.1 Prerequisites . . . 3

2.2 Compilation . . . 3

2.3 Running . . . 4

3. Input File Format **4**

4. Grid Configuration **5**

4.1 Cartesian Grid . . . 5

4.2 Grid Geometry . . . 5

4.3 Symmetry and Periodic Boundaries . . . 5

4.4 Distance Units . . . 5

4.5 Grid Type Selection . . . 6

5. Source Configuration **6**

5.1 Source Geometry . . . 6

5.2 Spectral Type . . . 7

5.3 Line Identification . . . 7

5.3.1 Combined Hydrogen + Deuterium Lyα . . . 8

5.3.2 He I 10833 Coherent Triplet Treatment . . . 9

5.4 Frequency and Wavelength Grid . . . 10

6. Medium Properties **11**

6.1 Optical Depth Normalization . . . 11

6.2 Temperature . . . 11

6.3 Density . . . 11

6.4 Velocity Fields . . . 12

7. AMR Grid Mode **13**

7.1 Enabling AMR Mode . . . 13

7.2 Generic AMR Data Format . . . 14

7.3 Extended AMR Physics Parameters . . . 15

7.4 Tau Normalization Convention . . . 15

7.5 Converting RAMSES Data . . . 15

7.6 Source Geometries in AMR Mode . . . 16

8. Clumpy Medium Mode **16**
8.1 Overview . . . 16
8.2 Enabling Clump Mode . . . 16
8.3 Clump Parameters . . . 17
8.4 Spatially-varying clump properties . . . 18
8.5 Standalone clump generator (`make_clumps.x`) . . . 18
8.6 Overlapping Clumps . . . 20
8.7 Clump Velocity . . . 21

9. Observers and Peel-off **21**
9.1 Observer Placement . . . 21
9.2 Multiple Observers . . . 21
9.3 Image Parameters . . . 22
9.4 Peel-off Output Flags . . . 22
9.5 Sight-line Optical-depth and Column-density Maps . . . 22
9.6 Inside Observer (HEALPix All-Sky) . . . 23

10. Output Files **24**
10.1 Primary Output Arrays . . . 25
10.2 Angular Spectrum `Jmu` . . . 25
10.3 Peel-off cube units and conversion to `Jmu` units . . . 26
10.4 Output Control Parameters . . . 27

11. File Formats: FITS and HDF5 **28**
11.1 Format selection . . . 28
11.2 HDF5 schema . . . 28
11.3 Conversion utility . . . 29
11.4 HDF5 library requirements . . . 29

12. Dust and Polarization **29**

13. Parallelism **30**

14. Complete Parameter Reference **30**

15. Python Utilities **35**
15.1 Loading and analysing output: `read_lart.py` . . . 35
15.2 Format-agnostic reader and FITS $\leftrightarrow$ HDF5 converter: `lart_io.py` . . . 38

16. Example Input Files **38**
16.1 Uniform Sphere, Lyα, Static . . . 38

16.2 Sphere with Hubble Outflow and Peel-off 39
16.3 Realistic Galaxy (FITS Density Field, Stokes Polarization) 39
16.4 AMR Sphere (Generic Text Format) 40
16.5 Clumpy Sphere with Random Velocities 40

1. Introduction

LaRT (**La**yman-alpha **R**adiative **T**ransfer) is a Monte Carlo radiative transfer code for computing the propagation of resonance-line photons through astrophysical neutral-gas media. The default transition is Lyman-α (Lyα, H I 1216 Å), but many other UV/optical/IR lines are supported (see §5.3). Dust scattering and absorption, full Stokes polarization tracking, Hubble-flow and other bulk velocity fields, and spatially varying density, temperature, and velocity from external data files are all supported.
Starting from v2.00, the Cartesian grid is supplemented by:

- **AMR mode** — octree adaptive mesh refinement, allowing spatially varying resolution (§7.).
- **Clumpy medium mode** — a population of compact, optically thick spherical clumps embedded in a transparent inter-clump medium (§8.).

This manual covers v2.00 and v2.10. The only user-visible change in v2.10 is the introduction of `par%grid_type` as a unified selector for the three grid modes; the old `par%use_amr_grid` and `par%use_clump_medium` flags remain valid for backward compatibility.

2. Installation and Build

2.1 Prerequisites

- Fortran 2003+ compiler: Intel `ifort/ifx` or GNU `gfortran` ≥ 7.
- CFITSIO library.
- HDF5 library (Fortran bindings), version ≥ 1.14, built with the same compiler used for LaRT. Required for the default build (`HDF5=1`); skip with `HDF5=0` for a FITS-only build.
- MPI library (MPICH or OpenMPI).

2.2 Compilation

```
cd LaRT_v2.00             # or LaRT_v2.10
make                      # produces LaRT.x (HDF5=1 is the default)
make HDF5=0               # FITS-only build (no HDF5 link dependency)
make HDF5_PREFIX=/path    # override HDF5 install location
```

```
make DEBUG=1            # full debug flags + runtime checks
make F90=mpif90         # override compiler
make clean              # remove .o and .mod files
make cleanall           # also remove executables
make convert_ramses     # auxiliary RAMSES converter
```

Every make invocation recompiles all objects (no incremental builds). par%HDF5_PREFIX defaults to /data/opt/hdf5_intel on this machine; set it on the command line or as an environment variable when building elsewhere.

2.3 Running

```
mpirun -np 8 LaRT.x input.in
```

Output is written to <base>.h5 by default (or <base>.fits.gz when par%file_format = 'fits' is set), where <base> is the input file base name (without .in) unless par%out_file is set explicitly. See §11. for the two supported file formats and how to choose between them.

3. Input File Format

Input files use Fortran *namelist* syntax. All parameters are prefixed with par% and collected in a single &parameters block.

```
&parameters
 par%no_photons      = 1e6
 par%temperature     = 1.0e4      ! gas temperature [K]
 par%taumax          = 1.0e4
 par%nx = 101, par%ny = 101, par%nz = 101
 par%rmax            = 1.0
 par%source_geometry = 'point'
 par%spectral_type   = 'voigt'
 par%nxfreq          = 121
/
```

- String values must be enclosed in single quotes.
- Multiple parameters may appear on one line, separated by commas.
- Lines beginning with ! are comments.
- Unspecified parameters retain compiled defaults (listed in §14.).

4. Grid Configuration

4.1 Cartesian Grid

The simulation domain is a rectangular box centered at the origin. By default $x \in [-x_{\max}, +x_{\max}]$ (and similarly for y and z). The cell size is $\Delta x = 2x_{\max}/n_x$.

Parameter	Default	Description
`par%nx`, `par%ny`, `par%nz`	101	Number of cells per axis
`par%xmax`, `par%ymax`, `par%zmax`	1.0	Half-box size per axis (code units)
`par%rmax`	–999	Spherical boundary: density = 0 for $r > r_{\max}$
`par%rmin`	–999	Inner cavity: density = 0 for $r < r_{\min}$
`par%cone_opening`	0.0	Biconical outflow half-opening angle [degrees] along the z-axis. When > 0, density is

If `par%input_field` or `par%dens_file` is given, `(nx,ny,nz)` and `(xmax,ymax,zmax)` are read from the FITS header automatically.

4.2 Grid Geometry

`par%geometry` selects the spatial symmetry used for accumulating and saving the mean-intensity array J:

Value	J output geometry
`'sphere'` (default)	1-D radial profile $J(r)$
`'cylinder'`	2-D cylindrical $J(\rho, z)$
`'rectangle'`	3-D Cartesian cube $J(x, y, z)$
`'plane_atmosphere'`	1-D slab $J(z)$
`'spherical_atmosphere'`	1-D spherical shell $J(r)$

4.3 Symmetry and Periodic Boundaries

Parameter	Default	Effect
`par%xyz_symmetry`	`.false.`	Simulate only the $+x, +y, +z$ octant ($8\times$ RAM saving)
`par%xy_symmetry`	`.false.`	Simulate only the $+x, +y$ quadrant ($4\times$ RAM saving)
`par%xy_periodic`	`.false.`	Periodic in x, y: photons exiting one face re-enter the opposite

Symmetry options are disabled automatically when peel-off is active.

4.4 Distance Units

Parameter	Default	Description
`par%distance_unit`	`''`	Physical unit: `'kpc'`, `'pc'`, `'au'`, or `''` (dimensionless)
`par%distance2cm`	1.0	Alternatively: 1 code unit in cm

In dimensionless mode (`par%distance_unit = ''`), the gas density is determined entirely by `par%taumax` and the grid geometry; no physical unit is needed.

4.5 Grid Type Selection

Parameter	Default	Description
`par%grid_type`†	`'car'`	**v2.10+**: `'car'` (Cartesian), `'amr'`, or `'clump'`
`par%use_amr_grid`	`.false.`	**v2.00**: enable AMR (deprecated in v2.10)
`par%use_clump_medium`	`.false.`	**v2.00**: enable clump mode (deprecated in v2.10)

5. Source Configuration

5.1 Source Geometry

`par%source_geometry` controls the spatial distribution of photon emission:

Value	Description
`'point'`	Point source at (`par%xs_point`, `par%ys_point`, `par%zs_point`); default $(0,0,0)$
`'uniform'`	Uniform volume emissivity throughout the grid (or inside `par%source_rmax`)
`'uniform_sphere'`	Uniform within a sphere of radius `par%source_rmax`
`'uniform_xy'`	Uniform in the $z = 0$ plane
`'gaussian'`	Gaussian: $\exp(-r^2/2\sigma_r^2)\exp(-z^2/2\sigma_z^2)$; widths `par%source_rscale`, `par%source_zscale`
`'exponential'`	Exponential disc: $\exp(-r/r_s)\exp(-\lvert z\rvert/z_s)$; scales `par%source_rscale`, `par%source_zscale`
`'exponential_sphere'`	Exponential clipped to sphere of radius `par%source_rmax`
`'exponential_cylinder'`	Exponential clipped to a cylinder
`'sersic'` / `'ssh'`	Sérsic profile; index `par%sersic_m`, half-light radius `par%Reff`
`'diffuse_emissivity'`	3-D emissivity read from `par%emiss_file`
`'star_file'`	Individual point sources from text file `par%star_file` (columns: x, y, z, L)

Associated parameters:

Parameter	Default	Description
par%xs_point, par%ys_point, par%zs_point	0.0	Point source position (code units)
par%source_rmax	–999	Outer truncation radius for extended sources
par%source_rscale	–999	Radial scale length
par%source_zscale	0.0	Vertical scale height
par%sersic_m	4.0	Sérsic index
par%Reff	1.0	Effective (half-light) radius
par%comoving_source	.true.	Source photons co-moving with the local gas

5.2 Spectral Type

par%spectral_type sets the frequency distribution of emitted photons:

Value	Description
'monochromatic'	Delta function at line center ($x = 0$)
'voigt'	Voigt profile at the local gas temperature
'voigt0'	Voigt profile at the fixed temperature par%temperature0
'gaussian'	Gaussian with σ =par%gaussian_sigma_vel or FWHM=par%gaussian_FWHM_vel [$\mathrm{km\,s^{-1}}$]
'continuum'	Flat spectrum over [par%velocity_min, par%velocity_max]
'continuum+gaussian'	Fraction $f_{\rm line}$ as Gaussian (FWHM=par%gaussian_FWHM_vel), rest flat continuum; $f_{\rm line} = W_\lambda^v/(W_\lambda^v + \Delta v)$ where $W_\lambda^v = \mathrm{EW}/\lambda_0\, c$ and Δv is the velocity range. Requires par%EW_line and par%gaussian_FWHM_vel.
'line_prof_file'	Arbitrary profile read from par%line_prof_file

5.3 Line Identification

The atomic transition is selected with par%line_id (default: Lyα).

`par%line_id`	Transition	Wavelength
(default)	H I Lyα	1215.67 Å
`'ly_alpha_HD'`	H I + D I Lyα	1215.67 Å (see below)
`'CII_1334'`	C II (+ fluorescence)	1334.53 Å
`'CIV_1548'`	C IV doublet	1548.19 / 1550.77 Å
`'NV_1239'`	N V doublet	1238.82 / 1242.80 Å
`'OVI_1032'`	O VI doublet	1031.91 / 1037.61 Å
`'SiIV_1394'`	Si IV doublet	1393.76 / 1402.77 Å
`'NaI_D'`	Na I D doublet	5891.58 / 5897.56 Å
`'CaII_HK'`	Ca II H & K doublet	3934.78 / 3969.59 Å
`'MgII_2796'`	Mg II doublet	2796.35 / 2803.53 Å
`'AlII_1671'`	Al II singlet	1670.79 Å
`'SiII_1190'` / `'SiII_1193'`	Si II doublet (+ fluorescence)	1190.42 / 1193.29 Å
`'SiII_1260'`	Si II (+ fluorescence)	1260.42 Å
`'SiII_1304'`	Si II (+ fluorescence)	1304.37 Å
`'SiII_1527'`	Si II (+ fluorescence)	1526.71 Å
`'FeII_2250'`	Fe II (+ fluorescence)	2249.88 Å
`'FeII_2261'`	Fe II (+ fluorescence)	2260.78 Å
`'FeII_UV3'` / `'FeII_2344'`	Fe II UV3 multiplet	2344.21 Å
`'FeII_UV2'` / `'FeII_2383'`	Fe II UV2 multiplet	2382.76 Å
`'FeII_UV1'` / `'FeII_2600'`	Fe II UV1 multiplet	2600.17 Å
`'HeI_10833'`	He I triplet	10832.06 Å

The numerical `line%line_type` that controls the dispatch of `par%calc_voigt` / `par%do_resonance` (set internally by `setup_resonance_line`) takes the following values: *1* = singlet (one upward transition); *2* = doublet (two upward transitions sharing a single thermal width, e.g. C IV, N V, O VI, Na I D, Ca II H&K, Mg II, Si IV); *4* = one resonance plus one or more fluorescent decays (most Si II, Fe II UV3); *5* = multiple upward + multiple downward transitions, the most general case (Si II 1190/1193, Fe II UV1, Fe II UV2); *6* = three upward transitions plus one downward transition (He I 10833); *7* = combined H + D Lyα (`ly_alpha_HD`, two coexisting two-level resonance scatterers, no fine structure).

5.3.1 Combined Hydrogen + Deuterium Lyα

The line id `'ly_alpha_HD'` treats neutral hydrogen and deuterium Lyα as two coexisting two-level resonance scatterers in the same medium. The two transitions sit only 0.33 Å ($\sim$ 82 km/s) apart and have nearly identical atomic constants except for the atomic mass and rest wavelength. The deuterium-to-hydrogen number-abundance ratio is set by

Parameter	Default	Description
`par%D_to_H_ratio`	1.5×10^{-5}	$n_{\rm D}/n_{\rm H}$ (cosmic primordial)
`par%include_deuterium`	`.false.`	Convenience flag

Setting `par%include_deuterium = .true.` together with `par%line_id = 'ly_alpha'` promotes the run automatically to `'ly_alpha_HD'`, so existing input files can opt in without changing the line id.

Physical assumptions in this version (v1):

- Both H and D are treated as simple two-level resonance scatterers (`line_type=7`). Fine-structure splitting is *not* included for either species; if `par%fine_structure = .true.` is set together with `'ly_alpha_HD'`, the code emits a warning and disables fine structure.
- The phase function is the simple dipole ($E_1 = 1$, $E_2 = 0$, $E_3 = 1$), identical for both species.
- Oscillator strengths and damping rates are taken to be equal for H and D ($f_{\rm D} = f_{\rm H}$, $\Gamma_{\rm D} = \Gamma_{\rm H}$ to better than 0.1%). Mass and rest wavelength differences are fully accounted for in the Voigt evaluation and species selection.
- Photon `xfreq` is carried in H-frame Doppler units throughout. D-frame conversions happen on-the-fly in the combined Voigt and scattering routines via constants precomputed at line setup (`line%delta_nu_HD_Hz`, `line%ratio_Dfreq_HD`, `line%ratio_voigta_HD`). No per-cell deuterium arrays are stored.
- During a deuterium scattering event, the perpendicular thermal velocity components and the recoil shift are evaluated using the H atomic constants, which produces a small ($\sim 1/\sqrt{2}$) error in the re-emission shape from D-scattered photons. Because D scatters are rare at cosmic abundance and the dominant D feature (the absorption dip) depends on opacity rather than re-emission shape, this is acceptable for v1. Per-photon species tracking is also not yet implemented.

The Deuterium Lyα rest wavelength used is $\lambda_{\rm D} = 1215.337$ (NIST vacuum), ~ 0.33 blueward of $\lambda_{\rm H} = 1215.668$. In LaRT's velocity convention ($v = -c\,\Delta\lambda/\lambda_{\rm H}$, so negative v is blueshifted), the D Lyα line center sits at $v_{\rm D} \approx -82\,{\rm km/s}$. A reference example with five validation runs and a Jupyter notebook is provided under `examples/lya_HD/` (see `run.sh` and `inspect_HD.ipynb`). One of those runs is configured for direct comparison against Dijkstra, Haiman & Spaans (2006), ApJ 649, 14, Figure 3 (D/H = 3×10^{-5}, $\tau \sim 10^6$).

5.3.2 He I 10833 Coherent Triplet Treatment

The He I $\lambda 10830$ multiplet $2^3S_1 \rightarrow 2^3P^o_{0,1,2} \rightarrow 2^3S_1$ is a three-component resonance line (LaRT `line_type=6`). The three upper levels are split by only ~ 0.09 (P_1–P_2, $\sim 2.5\,{\rm km/s}$) and ~ 1.25 (P_0–P_2, $\sim 35\,{\rm km/s}$), so depending on the gas temperature the components may overlap and interfere.

LaRT offers two treatments of the polarizability parameters (E_1, E_2, E_3) that enter the scattering phase function, selected by the boolean `par%HeI_coherent`:

Toggle	(E_1, E_2, E_3) source
`par%HeI_coherent = .false.` (default, legacy)	Per-component incoherent constants $(0,1,0)$, $(\frac{1}{4},\frac{3}{4},\frac{1}{4})$, $(\frac{7}{20},\frac{13}{20},\frac{3}{4})$ for P_0, P_1, P_2 from Seon `scatter_matrix_v1.1` §9.4. Assigned channel-by-channel based on which upper state the photon was sampled into.
`par%HeI_coherent = .true.`	Frequency-dependent $E_1(\nu), E_3(\nu)$ from the Real-Φ polynomial form of Seon `scatter_matrix_HeI` memo (eqs. 29 & 30, bottom line), with $E_2 = 1 - E_1$.

The polynomial form is, with $D_m \equiv \nu - \nu_{m1}$ for $m = 0, 1, 2$:

$$E_1(\nu) = \frac{3D_2^2D_0^2 + 7D_0^2D_1^2 + 8D_2D_0D_1^2 + 18D_2D_0^2D_1}{4\left[D_2^2D_1^2 + 3D_2^2D_0^2 + 5D_0^2D_1^2\right]}, \tag{1}$$

$$E_3(\nu) = \frac{3D_2^2D_0^2 + 15D_0^2D_1^2 + 8D_2^2D_0D_1 + 10D_2D_0^2D_1}{4\left[D_2^2D_1^2 + 3D_2^2D_0^2 + 5D_0^2D_1^2\right]}, \tag{2}$$

$$E_2(\nu) = 1 - E_1(\nu). \tag{3}$$

The common denominator is a positive-definite sum of squares with the three distinct line centers, so the formula is well-defined at every frequency and recovers the v1.1 §9.4 incoherent values exactly at each line center ($E_1, E_3 = 7/20, 3/4$ at P_2; $1/4, 1/4$ at P_1; $0, 0$ at P_0). It is implemented in `compute_HeI_E_coherent` in `line_mod.f90` (with parallel branches in `scattering_amr.f90` and `line_clump_mod.f90`).

Upper-state selection probability is unchanged. The total cross-section is the incoherent 1:3:5 weighted Lorentzian sum because all J'–J' interference terms cancel in $W_{11} + 2W_{14}$ (the angular integration projects onto the $K = 0$ irreducible component). Both modes therefore sample the upper state with the same probability, so the photon-by-photon frequency redistribution is statistically identical. Differences between the two modes appear only in the phase-function asymmetry E_1 (linear Rayleigh polarizability) and the circular polarizability E_3.

Expected size of the effect. For nebular conditions ($T \sim 10^4$ K) the He-atom Doppler width $\Delta v_D \sim 6.4\,\mathrm{km/s}$ *exceeds* the P_1–P_2 splitting ($\sim 2.5\,\mathrm{km/s}$), so those two components are effectively merged and the inter-line interference is small in the line core. The escape spectrum is essentially unchanged between the two modes. Polarization differences are visible mainly at high τ ($\tau \gtrsim 100$) and in the line wings where photons sample the inter-line frequency range; integrated polarization profiles differ by ~ 5–$15\,\%$ (relative) at $\tau \sim 10^3$. The effect would be larger at lower temperature ($T \lesssim 10^3$ K), where the P_1 and P_2 components are resolved.

A reference test suite is provided under `examples/HeI_coherent_test/` with 20 runs covering {point, uniform-sphere source} $\times$ $\{\tau = 0.1, 1, 10, 100, 1000\}$ $\times$ {`.false.`, `.true.`} and a comparison notebook `compare.ipynb`.

5.4 Frequency and Wavelength Grid

Internally, photon frequencies are tracked in the dimensionless Doppler variable:

$$x = \frac{\nu - \nu_0}{\Delta\nu_D}, \qquad \Delta\nu_D = \frac{\nu_0}{c}\sqrt{\frac{2kT}{m}}, \tag{4}$$

where ν_0 is the line-center frequency, T is the local gas temperature, and m is the atomic/ionic mass. Output spectra are binned onto a uniform grid in x by default. A wavelength or velocity grid can be requested instead:

Parameter	Default	Description
`par%nxfreq`	121	Number of spectral bins
`par%xfreq_min`, `par%xfreq_max`	auto	Limits of x-grid
`par%xfreq0`	0.0	Frequency offset applied to all emitted photons
`par%nwavelength`	0	Number of wavelength bins (overrides x-grid)
`par%wavelength_min`, `par%wavelength_max`	—	[Å]
`par%nvelocity`	0	Number of velocity bins (overrides x-grid)
`par%velocity_min`, `par%velocity_max`	—	[$\mathrm{km\,s^{-1}}$]

The range is set automatically from the mean medium temperature when no limits are provided.

6. Medium Properties

6.1 Optical Depth Normalization

The overall opacity is scaled by one of the following (specify at most one):

Parameter	Description
`par%taumax`	Line-center τ_0 from the grid center to the $+z$ boundary along the z-axis
`par%tauhomo`	Same, assuming a uniform density field (scales an inhomogeneous field proportionally)
`par%N_HImax`	H I column density from center to $+z$ boundary [$\mathrm{cm^{-2}}$]
`par%N_HIhomo`	Same, assuming uniform density

For AMR grids, `par%taumax` is the pole-to-face optical depth (box center $\to$ $+z$ face along z), consistent with the Cartesian convention for a sphere of radius $R = L/2$.

6.2 Temperature

Parameter	Default	Description
`par%temperature`	10^4 K	Uniform gas temperature
`par%temperature0`	$= T$	Temperature for `spectral_type='voigt0'` source
`par%bturb`	0.0	Turbulent velocity dispersion added in quadrature to thermal width [$\mathrm{km\,s^{-1}}$]
`par%temp_file`	`''`	3-D temperature cube [K]; FITS or HDF5

6.3 Density

Analytic profiles. Without external files, the density is set analytically via the opacity scale parameters (`par%taumax` etc.) combined with an optional radial profile:

Profile type	Key parameter	Functional form
Uniform (sphere or box)	`par%rmax`	Constant $n_{\rm H}$ inside $r < r_{\max}$
Spherical shell	`par%rmin`, `par%rmax`	Constant $n_{\rm H}$ between $r_{\min}$ and $r_{\max}$
Exponential	`par%density_rscale`	$n \propto \exp(-r/r_s)$
Power law	`par%density_alpha`	$n(r) \propto (r_{\max}/r)^{\alpha_D}$; α_D=0: uniform, α_D=2: isothermal

External gridded data (FITS or HDF5).

The reader (`read_grid_data.f90`) detects the format from the filename extension (`.fits[.gz]`, `.h5`, or `.hdf5`) and uses the same code path for both. HDF5 inputs expect the array at `/<section>/data` (any section name); shape information is read from the dataset itself, so FITS-style `NAXIS` keywords are not required.

Parameter	Description
`par%dens_file`	3-D H I number density cube [cm^{-3}]
`par%temp_file`	3-D temperature cube [K]
`par%velo_file`	4-D velocity cube (v_x, v_y, v_z) [km s^{-1}]
`par%emiss_file`	3-D emissivity cube [photons s^{-1} cm^{-3}]
`par%input_field`	Base name: reads `<base>.dens.fits.gz`, `.temp.`, `.velo.` (legacy convention; FITS-only)

If `par%taumax` (or `par%N_HImax`) is specified alongside a density file, the density is rescaled so that the target optical depth is achieved.

6.4 Velocity Fields

`par%velocity_type` selects the bulk velocity model. The default (empty string) is a static medium.

Value	Key parameters	Velocity law
`'hubble'`	`par%Vexp`	$\mathbf{v}(r) = V_{\rm exp}(r/r_{\max})\hat{r}$; $V_{\rm exp} > 0$: outflow
`'constant_radial'`	`par%Vexp`	$\mathbf{v}(r) = V_{\rm exp}\hat{r}$ (constant magnitude)
`'power_law'`	`par%Vexp`, `par%velocity_alpha`	$\mathbf{v}(r) = V_{\rm exp}(r/r_{\max})^{\alpha_V}\,\hat{r}$
`'linear_decelerate'`	`par%Vexp`	$\mathbf{v}(r) = V_{\rm exp}\,\frac{r_{\max}-r}{r_{\max}-r_{\min}}\,\hat{r}$
`'parallel_velocity'`	`par%Vx`, `par%Vy`, `par%Vz`	Uniform $\mathbf{v} = (V_x, V_y, V_z)$ [km s^{-1}]
`'ssh'`	`par%Vpeak`, `par%rpeak`, `par%DeltaV`	Bilinear profile (Song, Seon & Hwang 2020)
`'rotating_solid_body'`	`par%Vrot`, `par%rinner`	Solid-body rotation
`'rotating_galaxy_halo'`	`par%Vrot`, `par%q`	Galaxy halo rotation curve

The 'ssh' profile rises linearly to $V_{\rm peak}$ at $r_{\rm peak}$, then shifts by ΔV toward $r_{\rm max}$:

$$v(r) = \begin{cases} V_{\rm peak}\, r/r_{\rm peak} & r < r_{\rm peak} \\ \left[V_{\rm peak} + \Delta V\, \frac{r - r_{\rm peak}}{r_{\rm max} - r_{\rm peak}}\right] \hat{r} & r \geq r_{\rm peak}. \end{cases} \tag{5}$$

A velocity FITS file (`par%velo_file`) may be combined with a systematic `par%velocity_type`.

Other velocity-related parameters:

Parameter	Default	Description
`par%Omega`	28.0	Shear rate [(km s^{-1}) kpc^{-1}] for TIGRESS data
`par%recoil`	`.false.`	Include photon-recoil frequency shift in scattering
`par%core_skip`	`.false.`	Enable resonance-core skipping acceleration
`par%core_skip_global`	`.false.`	With `core_skip`: use the old volume-averaged $x_{\rm crit}$ instead of the per-cell formula (benchmar

Core-skip details. When `core_skip = .true.` and `core_skip_global = .false.` (the default combination), LaRT recomputes the core-skip threshold $x_{\rm crit}$ at every scattering using the local cell:

$$a\tau_{\rm cell} = a_{\rm V}({\rm cell})\,\texttt{rhokap}({\rm cell})\,\Delta\ell_{\rm face}, \qquad x_{\rm crit} = \begin{cases} (a\tau_{\rm cell})^{1/3}/5 & a\tau_{\rm cell} > 1 \\ 0 & \text{otherwise} \end{cases}$$

where $\Delta\ell_{\rm face}$ is the photon's distance to the nearest cell boundary. This is Smith et al. (2015, Eq. 35) applied per cell, following the RASCAS convention. Inhomogeneous boxes such as a dense galactic core in a sparse hot halo require this per-cell treatment; the `core_skip_global = .true.` branch uses a single volume-averaged $x_{\rm crit}$ that collapses to zero whenever $\langle a\tau \rangle < 1$ and is retained only for comparison with the older LaRT behavior.

7. AMR Grid Mode

7.1 Enabling AMR Mode

v2.00:

```
par%use_amr_grid = .true.
par%amr_file     = 'grid.h5'
```

v2.10 (preferred):

```
par%grid_type = 'amr'
par%amr_file  = 'grid.h5'
```

LaRT reads only the *generic AMR format* (text, FITS, or HDF5). Simulation data from RAMSES or other codes must first be converted using the standalone converter (`convert_ramses_to_generic.x` or the Python equivalent). The parameter `par%amr_type` defaults to 'generic'; setting it to 'ramses' is no longer supported and produces an error directing the user to the converter.

Parameter	Default	Description
`par%amr_type`	`'generic'`	Only `'generic'` is supported
`par%amr_file`	`''`	Path to generic AMR file (`.h5`, `.fits.gz`, `.dat`)

7.2 Generic AMR Data Format

The generic AMR format stores leaf-cell data in text, FITS binary table, or HDF5. Nine columns are mandatory; six optional columns provide pre-computed physics quantities that LaRT can use directly.

Mandatory columns (9):

Column	Unit	Description
`x`, `y`, `z`	code units	Leaf cell center coordinates
`level`	—	AMR refinement level
`nH` (or `gasDen`, `dens`)	cm^{-3}	Total hydrogen number density
`T`	K	Gas temperature
`vx`, `vy`, `vz`	$\mathrm{km\,s}^{-1}$	Gas velocity

Optional columns (detected by name in HDF5/FITS):

Column	Unit	Description
`metallicity`	mass fraction	Raw Z from simulation
`xHI`	dimensionless	Neutral hydrogen fraction
`n_e`	cm^{-3}	Electron density
`n_ion`	cm^{-3}	Scattering ion density (metal lines)
`emissivity`	$\mathrm{cm}^{-3}\,\mathrm{s}^{-1}$	Lyα emissivity rate
`ndust`	cm^{-3}	Dust pseudo-number density

When an optional column is present in the file, LaRT uses it directly. When absent, LaRT computes the quantity internally using the model selected by the extended physics parameters (see below).

Plain text format: a header followed by one line per leaf cell.

```
# comment lines start with #
BOXLEN  100.0
NLEAF   8192
# x  y  z  level  nH  T  vx  vy  vz  metallicity  xHI
50.5  50.5  50.5  1  1.0e-3  1.0e4  0.0  0.0  0.0  0.01  0.95
...
```

Leaf-cell coordinates are given with the box origin at the corner, $[0, \mathrm{BOXLEN}]$. The reader internally re-centers to $[-L/2, +L/2]$; *do not* pre-shift the data.

7.3 Extended AMR Physics Parameters

These parameters control how LaRT computes physical quantities when the corresponding optional column is absent from the generic file. All defaults reproduce the existing behavior.

Parameter	Default	Description
`par%ionization_model`	`'cie_formula'`	Neutral fraction model: `'cie_formula'` (single-formula CIE), `'cie_table'` (Gnat & Sternberg 2007), `'full_neutral'` ($x_{HI} = 1$), `'from_file'` (require `xHI` column)
`par%dust_model`	`'global_dgr'`	Dust model: `'global_dgr'` (use `par%DGR`), `'laursen09'` (metallicity-based, Laursen et al. 2009), `'from_file'` (require `ndust` column)
`par%emissivity_model`	`'none'`	Lyα emissivity: `'none'`, `'caseB'` (Case B recombination + collisional excitation), `'from_file'`
`par%ion_model`	`'none'`	Metal-line ion density: `'none'`, `'solar_cie'` (solar abundances × CIE ion fractions), `'from_file'`
`par%metallicity_global`	-1	Global metallicity (Z, mass fraction). Used by `'laursen09'` and `'solar_cie'` when no metallicity column is present.
`par%Z_ref`	0.0134	Reference solar metallicity (Asplund et al. 2009)
`par%f_ion_dust`	0.01	Ionized-gas dust fraction (Laursen et al. 2009)

7.4 Tau Normalization Convention

For AMR, `par%taumax` is the line-center optical depth from the box center to the $+z$ face along the z-axis. This matches the Cartesian sphere convention where `par%taumax` is the center-to-surface optical depth, provided $r_{max} = L/2$.

7.5 Converting RAMSES Data

RAMSES binary snapshots must be converted to the generic format before use with LaRT. Two converters are provided:

Fortran:

```
make convert_ramses
convert_ramses_to_generic.x /path/to/ramses 91 output.h5 kpc
convert_ramses_to_generic.x /path/to/ramses 91 output.h5 kpc \
    --compute-physics --metallicity 0.0134
```

Python:

```
python convert_ramses_to_generic.py /path/to/ramses 91 -o output.h5 \
    --compute-physics --metallicity 0.0134
```

The `--compute-physics` flag pre-computes `xHI`, `n_e`, and `emissivity` using the CIE formula. If `--metallicity` is given, `ndust` is also computed via the Laursen et al. (2009) model.

7.6 Source Geometries in AMR Mode

In v2.00, AMR supports `'point'` and `'uniform'`. In v2.10, most Cartesian geometries (Gaussian, exponential, Sérsic, `star_file`, etc.) are also available; refer to §5.1 and the v2.10 release notes.

8. Clumpy Medium Mode

8.1 Overview

Clumpy medium mode places N spherical clumps of radius r_c inside a spherical outer domain of radius R. The inter-clump medium is vacuum. All opacity resides inside the clumps. By default, clump positions are drawn by Random Sequential Adsorption (RSA) so that clumps never overlap. Optionally, overlapping clumps can be generated or loaded from an external FITS file; LaRT detects overlaps automatically after placement and engages an event-based multi-component raytrace in that case (see §8.6).

8.2 Enabling Clump Mode

v2.00:

```
par%use_clump_medium = .true.
```

v2.10 (preferred):

```
par%grid_type = 'clump'
```

8.3 Clump Parameters

Parameter	Default	Description
par%rmax	1.0	Outer sphere radius (code units)
par%rmin	–999	Inner cavity radius (the default sentinel is treated as 0). Clumps are placed only in the shell $r_{\min} \le r \le r_{\max}$
par%clump_radius	—	Individual clump radius (same units)
par%clump_tau0	—	Line-center τ_0 from clump center to its surface
par%clump_f_cov	—	Covering factor $C_f = N r_c^2 / R^2$
par%clump_f_vol	—	Volume filling factor $f_V = N(r_c/R)^3$
par%clump_N_clumps	—	Total number of clumps N
par%temperature	10^4 K	Clump gas temperature
par%clump_sigma_v	0.0	Gaussian σ of random bulk clump velocities [km s^{-1}]
par%save_clump_info	.false.	Write clump positions and velocities to a FITS binary table
par%clump_input_file	''	If non-empty, load the clump population from this FITS file (see §8.5) instead of generating it internally
par%cone_opening	0.0	Biconical outflow half-opening angle [degrees]. When > 0, clumps are placed only inside the bicone (both hemispheres). With par%clump_fully_inside, clumps are additionally rejected if they protrude outside the cone boundary.
par%clump_fully_inside	.true.	If .true. (default), reject any candidate whose outermost edge would protrude past the outer sphere ($d_{\rm center} + r_c > R$) or into the inner cavity ($d_{\rm center} - r_c < r_{\min}$); if .false., restore the legacy behavior in which only the center is required to lie in $[r_{\min}, R]$
par%clump_allow_overlap	.false.	If .true., skip RSA overlap rejection during internal generation: clumps are placed at uniformly random positions. After placement check_has_overlap() is called automatically; if any overlapping pair is found the overlap-aware raytrace is engaged. See §8.6

Specify exactly one of par%clump_f_cov, par%clump_f_vol, or par%clump_N_clumps.

For the per-clump opacity, specify *one* of par%clump_tau0, par%clump_NHI, or par%clump_nH. As a convenience, when none of those three is set the system-level radial-sightline targets par%taumax and par%N_HImax are also accepted for procedurally generated clumps: LaRT back-solves the peak per-clump opacity so the realised radial sightline from the box centre to $r = R$ matches the requested par%taumax (or par%N_HImax). In the uniform-clump case the back-solve uses the closed-form $\tau_{\max} = \frac{4}{3} f_{\rm cov}\, \tau_{c,0}$ (and similarly for $N_{\rm HI,max}$); in profile mode it uses the radial quadrature derived from `par%clump_radius_profile`, `par%clump_density_profile`, and `par%clump_number_profile`. `par%N_gasmax` is treated as an alias for `par%N_HImax` in clump mode (HI-only line). When a pre-built clump population is loaded via `par%clump_input_file`, any one of `par%taumax`, `par%tauhomo`, `par%N_HImax` (= `par%N_gasmax`), or `par%N_HIhomo` (= `par%N_gashomo`) may be supplied as the rescale target. Every `cl_rhokap(i)` is multiplied by a single factor α so the chosen target is exactly reproduced; the other three scalars are derived consistently from the rescaled distribution. Priority order when several targets are given: `par%taumax` > `par%tauhomo` > `par%N_gasmax` > `par%N_gashomo`. See the *Clumpy Medium Algorithm Description*, §"System-level fallback".

When `par%rmin` is set, `par%clump_f_vol` and `par%clump_f_cov` are interpreted relative to the shell volume $V_{\rm shell} = \frac{4}{3}\pi(R^3 - r_{\rm min}^3)$ and the shell sightline factor $R^2 + R\,r_{\rm min} + r_{\rm min}^2$. With `par%rmin` unset (default –999, treated as 0) these reduce exactly to the original $(R/r_c)^3$ and $(R/r_c)^2$ formulas, preserving prior runs. When `par%clump_fully_inside` is on, the realized filling factors fall slightly below the requested target whenever r_c is a non-trivial fraction of $R - r_{\rm min}$, since the available placement volume shrinks from $\frac{4}{3}\pi(R^3 - r_{\rm min}^3)$ to $\frac{4}{3}\pi\left[(R-r_c)^3 - (r_{\rm min}+r_c)^3\right]$. Specify a slightly larger target if exact matching is required. See the *Clumpy Medium Algorithm Description* document, "Fully-Inside Constraint" subsection, for the placement details.

8.4 Spatially-varying clump properties

Three optional radial-profile inputs let the per-clump radius, internal H I density, and spatial number density vary with distance from the box center. Built-in shapes are `'constant'` (default), `'powerlaw'`, `'gaussian'`, `'exponential'`, and `'file'` (5-column ASCII table). Setting any flag to a value other than `'constant'` switches `par%init_clumps` to the profile-aware path.

Parameter	Default	Description
`par%clump_radius_profile`	`'constant'`	shape of $r_c(r)$
`par%clump_radius_alpha`	0.0	power-law index for $r_c(r)$
`par%clump_radius_r0`	0.0	power-law scale for $r_c(r)$; $\le 0 \Rightarrow r_{\rm max}$
`par%clump_density_profile`	`'constant'`	shape of internal $n_H(r)$
`par%clump_density_alpha`	0.0	power-law index for $n_H(r)$
`par%clump_density_r0`	0.0	power-law scale for $n_H(r)$; $\le 0 \Rightarrow r_{\rm max}$
`par%clump_number_profile`	`'constant'`	shape of spatial $n_{cl}(r)$
`par%clump_number_alpha`	0.0	power-law index for $n_{cl}(r)$
`par%clump_number_r0`	0.0	power-law scale for $n_{cl}(r)$; $\le 0 \Rightarrow r_{\rm max}$
`par%clump_profile_file`	`''`	5-column tabulated profile (used when any axis = `'file'`)

For non-uniform profiles the closed-form $f_{\rm cov}$ / $f_{\rm vol}$ / N relation does not hold. Instead the profile-aware quadrature back-solves the spatial normalization so that whichever of `par%clump_f_cov`, `par%clump_f_vol`, or `par%clump_N_clumps` is provided is matched, and the other two are reported on stdout. See the *Clumpy Medium Algorithm Description* document for the integral definitions and the implementation outline.

8.5 Standalone clump generator (make_clumps.x)

The same Fortran routines that build the clump population inside LaRT are also exposed through a small standalone driver, make_clumps.x:

```
make make_clumps                          # builds make_clumps.x
mpirun -np N ./make_clumps.x clumps.in    # writes <base>_clumps.fits.gz
```

The driver reads the same namelist as LaRT.x and writes the realized clump population to <base_name>_clumps.fits.gz without running the photon transport. Fix par%iseed to a non-zero value for reproducibility: under the same seed the table produced by make_clumps.x matches the _clumps.fits.gz written by a regular LaRT.x run with par%save_clump_info = .true. byte-for-byte.

To consume the file, set par%clump_input_file on the LaRT input:

```
&parameters
 par%grid_type         = 'clump'
 par%rmax              = 1.0
 par%clump_radius      = 0.01
 par%clump_tau0        = 1.0e3
 par%temperature       = 1.0e4
 par%no_photons        = 1e6
 par%clump_input_file = 'clumps_clumps.fits.gz'
/
```

When par%clump_input_file is non-empty, init_clumps skips the profile-detection and RSA-generation paths entirely and reads the population from the file. read_clumps_fits accepts files with any subset of the per-clump columns R_CLUMP/RHOKAP/TEMP; missing columns are filled from the corresponding header keyword (CL_RAD/RHOKAP/TEMP_CL). For the RHOKAP slot the column names DENSITY and DENS are also accepted as aliases, in priority order RHOKAP $\rightarrow$ DENSITY $\rightarrow$ DENS. The interpretation depends on which name was used: RHOKAP (and the header-keyword fallback) is taken as line-centre opacity per code unit and stored directly into κ_i, while DENSITY or DENS are taken as the neutral-hydrogen number density $n_{HI,i}$ [cm^{-3}] and converted to opacity via $\kappa_i = n_{HI,i} \cdot \texttt{line\%cross0}/\Delta\nu_{D,i} \cdot \texttt{par\%distance2cm}$. The conversion requires par%distance_unit (and therefore par%distance2cm) to be set; if not, the read path emits a warning and falls back to treating the values as opacity.

The clump file also carries DISTUNIT (par%distance_unit) and DIST_CM (par%distance2cm) keywords written by LaRT itself. On read, these are adopted into par%distance_unit / par%distance2cm when the user did NOT supply them in the input namelist; the criterion for “user-supplied” is par%distance_unit /= '' *and* par%distance2cm > 1.0, so the file values take effect only when the namelist leaves both at their defaults. This lets a clump file generated with par%distance_unit = 'kpc' be replayed in a downstream run without having to repeat that setting, and gives a meaningful par%distance2cm to the DENSITY/DENS opacity conversion above. External pipelines that write LaRT-compatible HDF5 clump files must store DISTUNIT as a fixed-length ASCII string (h5py: string_dtype(encoding='ascii', length=N)); variable-length strings are detected on read and silently skipped (a warning is printed).

If any of `par%taumax`, `par%tauhomo`, `par%N_HImax` (=`par%N_gasmax`), or `par%N_HIhomo` (=`par%N_gashomo`) is specified together with `par%clump_input_file`, every loaded `cl_rhokap(i)` is multiplied by a single factor α = target/realised so the chosen system-level scalar matches the input target; the other three scalars are then derived consistently from the rescaled population. Priority among multiple targets: `par%taumax` > `par%tauhomo` > `par%N_gasmax` > `par%N_gashomo`. If none of the four is set, the loaded opacities are taken as-is and the four scalars are reported as computed from the file. See *Clumpy Medium Algorithm Description*, §"System-level fallback" and §"Derived Global Optical-Depth Quantities" for the formulas.

8.6 Overlapping Clumps

By default LaRT guarantees non-overlapping clumps through RSA placement and all raytrace paths assume this property. Two situations can produce overlapping populations:

1. A clump population loaded from an external FITS file via `par%clump_input_file` may contain overlapping clumps.
2. Setting `par%clump_allow_overlap = .true.` skips the RSA rejection step so that internally generated clumps are placed at uniformly random positions and will typically overlap whenever the clumps are large relative to the domain.

After the CSR acceleration grid is built, `check_has_overlap()` scans all clump pairs. If any overlapping pair is found it prints

```
Overlap check: overlapping clumps detected -- using overlap-aware raytrace.
```

sets an internal flag `has_overlap = .true.`, and routes all subsequent raytrace calls to the overlap-aware variants automatically. No input change is required; the detection and dispatch happen at run time.

Physical model for overlapping clumps

Two clumps sharing a volume with different bulk velocities $\mathbf{v}_1$ and $\mathbf{v}_2$ are treated as two independent gas components coexisting at the same spatial location. The total line opacity at global-frame frequency x is

$$\kappa_{\rm total}(x) = \sum_i \rho\kappa_i H(x - u_{{\rm los},i}, a_i), \qquad u_{{\rm los},i} = \frac{\mathbf{v}_i \cdot \hat{k}}{\Delta\nu_{D,i}}. \tag{6}$$

When a scattering event occurs in an overlap region the interacting atom belongs to one clump, chosen by opacity-weighted sampling $P(\text{clump } i) = \kappa_i/\kappa_{\rm total}$.

Quick-start example

Large clumps with low covering factor guarantee many overlaps while keeping the run fast:

```
par%grid_type            = 'clump'
par%clump_radius         = 0.3
par%clump_f_cov          = 2.0
```

```
par%clump_allow_overlap = .true.
par%no_photons          = 1e5
par%spectral_type       = 'voigt'
```

Reference inputs: `clump_overlap_mono.in`, `clump_overlap_sigv.in`, `clump_overlap_peel.in` in `examples/clump_sphere/`.

For a full algorithmic description see the *Clumpy Medium Algorithm Description* document, Section 15 “Overlap-Aware Raytrace”.

8.7 Clump Velocity

A systematic flow can be added on top of the random component (`par%clump_sigma_v`) via `par%velocity_type`. Supported values in clump mode: `'hubble'`, `'constant_radial'`, `'power_law'`, `'linear_decelerate'`, `'parallel_velocity'`, `'ssh'`, `'rotating_solid_body'`, `'rotating_galaxy_halo'`.

9. Observers and Peel-off

The peel-off (peeling-off) technique shoots a photon toward the observer at every scattering event, allowing spectral images to be constructed without noise. It is activated by setting `par%nxim` and `par%nyim` to non-zero values.

9.1 Observer Placement

Direction vector:

```
par%obsx = 0.0, par%obsy = 0.0, par%obsz = 1.0  ! look down +z axis
par%distance = 100.0                               ! observer-center separation
```

Spherical angles:

```
par%alpha = 45.0   ! azimuthal angle [deg]
par%beta  = 30.0   ! elevation angle [deg]
par%distance = 100.0
```

`par%gamma` sets the rotation of the detector plane about the line of sight. If `par%distance` is omitted it defaults to $100 \times r_{\rm max}$ (or the maximum box half-extent).

9.2 Multiple Observers

Multiple observers are specified as arrays:

```
par%alpha = 0.0  45.0  90.0  135.0
par%beta  = 0.0   0.0   0.0    0.0
```

Up to 99 simultaneous observers are supported.

9.3 Image Parameters

Parameter	Default	Description
par%nxim, par%nyim	0	Image pixel counts; peel-off enabled when > 0
par%dxim, par%dyim	auto	Pixel angular size [deg] (defaults to cover the full grid)

When par%dxim/par%dyim are not set, the pixel size is chosen to cover the full sphere:

$$\delta\theta = \frac{1}{N_x/2} \arcsin\left(\frac{r_{\max}}{d}\right). \tag{7}$$

9.4 Peel-off Output Flags

Parameter	Default	Description
par%save_peeloff	.false.	Save 1-D peel-off spectra per observer
par%save_peeloff_2D	.false.	Save peel-off spectral image cubes (x-y-ν)
par%save_peeloff_3D	.false.	Save full 3-D peel-off data
par%save_sightline_tau	.false.	Compute deterministic sight-line maps from each observer pixel: gas $\tau(\nu)$, $N_{\rm HI}$, and dust τ (when par%DGR>0). See §9.5.

9.5 Sight-line Optical-depth and Column-density Maps

When par%save_sightline_tau = .true., LaRT integrates the opacity along each observer pixel's line of sight and writes a separate FITS file <base>_tau<obs>.fits.gz for every observer. The file contains:

HDU	Content
TAU_gas	gas optical depth $\tau(\nu, x, y)$
N_gas	gas column density $N_{\rm HI}(x, y)$
TAU_dust	dust optical depth $\tau_{\rm dust}(x, y)$ (only when par%DGR>0)

The pixel grid uses the same WCS as the peel-off image (par%nxim, par%nyim, par%dxim, par%dyim); the frequency axis matches the spectrum output (par%nxfreq, par%velocity_min, par%velocity_max). All three grid modes (Cartesian, AMR, clump) are supported with the appropriate raytrace dispatch.

Standalone driver make_sightline_tau.x

The same sight-line integration is exposed as a standalone executable that skips the photon-transport stage entirely:

```
make sightline_tau                              # builds make_sightline_tau.x
mpirun -np N ./make_sightline_tau.x sl.in       # writes <base>_tau.fits.gz
```

sl.in is an ordinary LaRT input file; the driver internally sets par%save_peeloff and the sight-line flag to .true. and otherwise honors every namelist parameter. Useful for parameter sweeps where only the deterministic maps are needed (no Monte-Carlo transport). Unlike `make_clumps.x`, multiple MPI ranks do speed this driver up: pixels are divided across `par%nproc` via `loop_divide` and reduced with `reduce_mem`.

The folder `examples/sightline_tau/` contains a reference set of six runs that exercise every grid mode and observer configuration the driver supports:

File	Grid mode	Observer
`car_sightline.in`	Cartesian	outside (rectangular FITS)
`amr_sightline.in`	AMR	outside (rectangular FITS)
`clump_sightline.in`	clumpy sphere	outside (rectangular FITS)
`car_inside.in`	Cartesian	inside (HEALPix all-sky)
`amr_inside.in`	AMR	inside (HEALPix all-sky)

The shipped `run.sh` executes all six in sequence; `make_amr_sphere_data.py` regenerates the AMR data file from a uniform-sphere physics model. The companion notebook `plot_sightline_tau.ipynb` displays the column-density maps, selected $\tau(x, y, v)$ slices, the all-sky HEALPix mollview maps and the sky-mean line profile.

tau_huge cap (sight-line vs. photon transport). The AMR raytrace routines used by the photon-transport stage early-exit when the running optical depth exceeds the double-precision exp($-\tau$) underflow point ($\tau_{\rm huge} \approx 745.2$); accumulating further contributes nothing to the photon's transmission. The sight-line variant `raytrace_to_edge_tau_gas_amr` exists precisely to *report* the integrated τ, however large, and therefore runs without that cap (matching the Cartesian `raytrace_to_edge_tau_gas` in `raytrace_car.f90`). The same applies to `raytrace_to_dist_tau_gas_amr` used by the inside-observer HEALPix path.

9.6 Inside Observer (HEALPix All-Sky)

Setting `par%nside` > 0 switches the entire output and peel-off chain to the *inside-observer* path: the observer is placed at a chosen position (o_x, o_y, o_z) *inside* the medium, and each escaping photon is binned into an all-sky HEALPix map by the direction from the observer back to the photon. This mode is intended for "galactic" observations – e.g., placing the observer at the Sun's position inside a model of the Galactic ISM, or inside a simulated galaxy.

Activation:

```
par%nside = 8                    ! HEALPix Nside parameter; total npix = 12*nside^2
par%obsx  = 0.3                  ! observer position (code units)
par%obsy  = 0.0
par%obsz  = 0.0
par%save_peeloff    = .true.
par%save_peeloff_2D = .true.     ! also save the 2D map (sum over frequency)
```

When `par%nside` > 0, `setup.f90` automatically sets `par%observer_located_inside = .true.` and switches:

- the observer arrays from rectangular images to HEALPix all-sky maps with $N_{\rm pix} = 12\,{\rm Nside}^2$;
- the peel-off routines to the `_inside` variants (which use `vec2pix` to bin the peeled weight by observer-pointing direction);

- `output_reduce`, `output_normalize` and `write_output` to the `_inside` variants.

In v2.10 the dispatch is performed by the type-bound methods of `grid_base_type` (`grid_class.f90`); the `grid_car_type` and `grid_amr_type` concrete types each branch on `par%observer_located_inside` and forward to the appropriate `_inside` or `_outside` concrete routine.

Multiple observers. As with the rectangular path, up to 99 observer positions are supported by passing arrays:

```
par%nside = 8
par%obsx  = 0.0  0.3  0.0
par%obsy  = 0.0  0.0  0.5
par%obsz  = 0.0  0.0  0.0
```

Output files. In inside-observer mode the `_obs.fits.gz` file holds 2D HEALPix arrays of shape `(npix, nxfreq)` with extensions `Direct` (unscattered direct contribution) and `Scattered` (scattered contribution). When `par%save_peeloff_2D = .true.` a separate `_obs2D.fits.gz` file is written with shape `(npix,)` holding the same arrays summed over frequency. The HEALPix maps use the RING ordering convention. The `Spectrum` HDU (`Jout`, `Jin`, `Jabs`) is always written regardless of observer mode.

Source scope. The inside-observer path supports the same internal sources as the standard path – `point`, `uniform`, `gaussian`, `exponential`, `sersic`, `diffuse_emissivity` – but *not* the external-illumination geometries (`point_illumination`, `stellar_illumination`, `plane_illumination`), which are only physically meaningful for an observer outside the medium. Stokes polarization is not yet implemented for the inside-observer path.

Grid-mode support. The HEALPix inside-observer path is available for both Cartesian (`par%grid_type = 'car'`) and AMR (`par%grid_type = 'amr'`) grids. In AMR mode, dedicated `_inside_amr` peeling, output and sightline routines are used (see the *LaRT AMR Description* document for implementation details). Clumpy medium (`par%grid_type = 'clump'`) is not currently supported with the inside-observer path.

Sightline-tau output. When the inside-observer path is active, an additional `_sightline.fits.gz` file is produced that records τ and N(HI) from the observer position to each of the six box faces ($\pm x$, $\pm y$, $\pm z$). This is useful for cross-checking the optical depth structure seen by the observer.

Smoke test. The `examples/amr_sphere_generic/` directory ships `sphere_amr_inside_test.in` (AMR uniform sphere) and `sphere_car_inside_test.in` (Cartesian 64^3 counterpart), both with `par%nside = 8` and an off-center observer at $(o_x, o_y, o_z) = (0.3, 0, 0)$. These produce equivalent HEALPix maps within the documented AMR octree volume-averaging tolerance.

10. Output Files

Outputs are written either as gzip-compressed FITS files (`.fits.gz`) or as HDF5 files (`.h5`). HDF5 is the default; set `par%file_format = 'fits'` in the namelist to switch. Both formats share the same logical layout (one section per FITS HDU $\leftrightarrow$ one HDF5 group), so the conversion is loss-less and runs in both directions via `python/lart_io.py` (see §11. for the schema and converter, §15. for the Python helper).

10.1 Primary Output Arrays

Array	FITS extension	Description
`Jout`	`JOUT`	Emergent spectrum integrated over all escape directions
`Jin`	`JIN`	Injected (intrinsic) source spectrum
`Jabs`	`JABS`	Spectrum of photons absorbed by dust
`Jmu`	`JMU`	Emergent spectrum binned by $\mu = \cos\theta_z$

Jout is the main science output. It is normalized per unit frequency interval and per unit area of the bounding sphere. In dimensionless units (no `par%distance_unit`):

$$J_{\rm out}(x) = \frac{\text{escaped photon weight in bin } x}{n_{\rm ph}\ \Delta x\ 2\pi\ \pi R^2}, \tag{8}$$

where $R = r_{\rm max}$ (Cartesian) or $R = L_{\rm box}/2$ (AMR), and $\Delta x = 1$ in units of the Doppler width.

Jin requires `par%save_Jin = .true.`

Jabs requires `par%save_Jabs = .true.`

Jmu requires `par%save_Jmu = .true.`; see §10.2.

The total flux is conserved: $\int J_{\rm out}\, dx + \int J_{\rm abs}\, dx = \int J_{\rm in}\, dx$ (in the absence of other losses).

10.2 Angular Spectrum `Jmu`

Setting `par%save_Jmu = .true.` adds a 2-D image extension `EXTNAME='Jmu'` of shape (n_x, n_μ) that records the emergent spectrum binned by the z-component of the escape direction, $\mu \equiv \cos\theta_z = \hat{k}_z$. This is useful for

- verifying isotropy of homogeneous models (every μ bin should match `Jout` within Poisson noise),
- characterizing the angular dependence of escape in non-isotropic models (for example, a slab or rotating halo).

Binning.

- `par%nmu` bins of width $\Delta\mu$ tile the μ range uniformly.
- Default `par%nmu` = 11 (odd) places $\mu = 0$ at a bin center, so the perpendicular-escape spectrum is preserved as a single channel. Even `par%nmu` places $\mu = 0$ at a bin edge and cleanly separates the $+z$ and $-z$ hemispheres.
- Range:
  - `par%xyz_symmetry = .true.` folds escapes onto the $+z$ hemisphere, so $\mu \in [0, +1]$ and $\Delta\mu = 1/n_\mu$.
  - Otherwise $\mu \in [-1, +1]$ and $\Delta\mu = 2/n_\mu$.

  The header keywords `nmu`, `mu_min`, `dmu` record these values, and full WCS keywords (`CTYPE1='XFREQ'`, `CTYPE2='MU'`, `CRVAL*`, `CDELT*`) are written to the image header.

Normalization. `Jmu` uses the same denominator as `Jout` multiplied by n_μ, so that

$$\frac{1}{n_\mu}\sum_{i=1}^{n_\mu} J(x;\mu_i) = J_{\rm out}(x) \tag{9}$$

holds exactly. Equivalently, in a homogeneous, isotropic medium each μ bin reproduces `Jout` up to Poisson fluctuations, providing a cheap regression test for new geometries.

Modes. Available with all three grid modes: Cartesian, AMR octree, and the clumpy medium.

10.3 Peel-off cube units and conversion to `Jmu` units

When peel-off is enabled (`par%nxim`, `par%nyim` > 0 and `par%save_peeloff_3D = .true.`), each observer's `<base>_obs_NNN.fits.gz` file contains `Scattered` and `Direct` 3-D image cubes, plus optionally `Direct0`. These are written by `output_sum_rect.f90` as a *specific intensity per pixel*, normalized so that

$$I_{\rm peel}(x,y,\nu) \;=\; \frac{\text{photon weight in pixel }(x,y)\text{ and bin }\nu}{n_{\rm ph}\,\delta\Omega_{\rm pix}\,\Delta\nu\,d_{\rm cm}^2}, \tag{10}$$

where $\delta\Omega_{\rm pix} = (\texttt{CD2\_2}\cdot\texttt{CD3\_3})(\pi/180)^2$ is the per-pixel solid angle, $\Delta\nu$ is `Dxfreq` or `Dwave` (set by `par%intensity_unit`), $d_{\rm cm}^2$ is the square of the distance unit in cm, and $n_{\rm ph}$ is the photon count.

Conversion to `Jmu` units. `Jout` and `Jmu` are normalized by $4\pi R^2$ (the source surface area) and the outward solid angle 2π, whereas the peel-off cube is normalized by the per-pixel solid angle and $d_{\rm cm}^2$. Summing the cube over pixels and rescaling brings the two onto a common axis:

$$J_{\rm peel}(\nu) \;=\; \left(\sum_{(x,y)} I_{\rm peel}(x,y,\nu)\right)\cdot\frac{4\pi D^2\,\delta\Omega_{\rm pix}}{A_{\rm surface}\cdot 2\pi}, \tag{11}$$

where D is `DISTANCE` (in code units) and $A_{\rm surface}$ is the source surface area in code units, with the geometry-dependent forms mirroring `output_sum_rect.f90`:

Geometry	$A_{\rm surface}$ (code units)	Notes
sphere	$4\pi r_{\rm max}^2$	default; clumpy medium also uses this
box (rectangle)	$8\,(x_{\rm max}\,y_{\rm max}+y_{\rm max}\,z_{\rm max}+z_{\rm max}\,x_{\rm max})$	full surface of $[-x_{\rm max},x_{\rm max}]\times\ldots$ box
slab (`par%xy_periodic`)	$2\,(2x_{\rm max})(2y_{\rm max})$	two unbounded boundaries
AMR octree	4π	equivalent to $r_{\rm max}=1$ (cf. §7.)

The bin width $\Delta\nu$ cancels because both `Jmu` and `I_peel` are already normalized per bin. At convergence and for an axisymmetric source,

$$J_{\rm peel}(\nu;\alpha,\beta) \;\to\; J_{\rm mu}(\nu;\mu=\cos\beta). \tag{12}$$

The Python helper `LaRTOutput.plot_peel_jmu_compare()` (in `python/read_lart.py`) implements Eq. (11) with the surface area chosen automatically from `par%geometry`, `par%rmax`, and the spectrum-header `XMAX/YMAX/ZMAX/XY_PER`. See also `examples/sphere/plot_Jpeel_conversion.ipynb` and `examples/clump_sphere/verify_peel_jmu.ipynb`.

Component decomposition (clumpy medium). For a point source inside a clumpy sphere with covering factor $f_{\rm cov}$, the fraction of photons that escapes without scattering is $\exp(-f_{\rm cov})$ (Poisson statistics), and the fraction that scatters at least once is $1-\exp(-f_{\rm cov})$. Jmu tallies *both* populations. The peel-off cube splits them:

- `Scattered`. The MC estimator over many scatter events averages efficiently, so
$$\int J_{\rm peel}^{\rm scatt}\,d\nu \;\approx\; \left(1-e^{-f_{\rm cov}}\right)\int J_{\rm mu}\,d\nu, \tag{13}$$
and the equality holds across all observer angles within Monte-Carlo noise.
- `Direct`. For a *point* source, every photon shares the same line of sight to a given observer, so the direct contribution is the binary realization of $\exp(-\tau_{\rm LOS})$: either ~ 1 (clear LOS) or ~ 0 (blocked by an intervening clump) per observer. Jmu, by contrast, is azimuth averaged and captures direct escapes uniformly. Adding the binary `Direct` and the smooth `Scattered` therefore only matches `Jmu` *in expectation* – it can deviate substantially per observer at finite photon counts.

For shape comparisons in clumpy runs, prefer the scattered component (`component='scatt'` in `plot_peel_jmu_compare()`) so that Eq. (13) provides a clean, deterministic relation.

10.4 Output Control Parameters

Parameter	Default	Description
`par%file_format`	`'hdf5'`	Output container: `'hdf5'` (→ `.h5`) or `'fits'` (→ `.fits.gz`); see §11.
`par%out_file`	`''`	Output file name (defaults to `<input>.h5`; extension is corrected to match `par%file_format` with a warning)
`par%out_merge`	`.false.`	Accumulate into existing output instead of overwriting
`par%save_backup`	`.false.`	On merge, copy the prior output to `<base>_NN.{h5,fits.gz}` before overwriting
`par%out_bitpix`	0	Storage precision: 0 = auto (dynamic-range driven), −32 = float32, −64 = float64
`par%save_Jin`	`.false.`	Save injected spectrum
`par%save_Jabs`	`.false.`	Save dust-absorbed spectrum
`par%save_Jmu`	`.false.`	Save angular spectrum $J(x;\mu)$ (§10.2)
`par%nmu`	11	Number of μ bins for `Jmu`
`par%save_all`	`.false.`	Save full 3-D $J(\nu,x,y,z)$ array
`par%save_direc0`	`.false.`	Save photon escape direction distributions
`par%save_radial_profile`	`.false.`	Save 1-D radial J profile
`par%save_sightline_tau`	`.false.`	Save optical depth along sightlines
`par%intensity_unit`	`''`	Physical unit label written to the FITS header

`par%out_merge = .true.` is useful for noise reduction: run the same setup multiple times and merge the results. All input parameters must be identical across runs.

11. File Formats: FITS and HDF5

LaRT writes one logical layout that lives in either FITS or HDF5 files. The choice is made by par%file_format (default 'hdf5'); the filename extension is corrected to match if the user-supplied par%out_file disagrees.

11.1 Format selection

- par%file_format = 'hdf5' (default) writes .h5 files via the hdf5io_mod backend. Build with HDF5=1 (the Makefile default).
- par%file_format = 'fits' writes .fits.gz files via the legacy fitsio_mod backend (CFITSIO). Works regardless of the HDF5 build option.

The mismatch warning is issued once on rank 0, e.g. if par%out_file = 'foo.fits.gz' is given alongside par%file_format = 'hdf5':

```
WARNING: par%out_file (foo.fits.gz) does not match
         par%file_format = hdf5
         Overriding par%out_file to: foo.h5
         (set par%file_format explicitly to suppress this warning.)
```

All derived filenames (peel-off _obs, sight-line _tau, _allph, backup) follow par%file_format as well, so the main output and its derived files always share a format.

11.2 HDF5 schema

Each FITS HDU maps to one HDF5 group; the HDU's header keywords become attributes on that group; the HDU's data is either a single /<name>/data dataset (image-style) or one dataset per BinTable column (table-style). Example for a sphere run with peel-off:

```
run.h5
  /Spectrum                                       (group, table-style)
    /Spectrum/Xfreq         (1-D dataset, float32)
    /Spectrum/velocity      (1-D dataset, float32)
    /Spectrum/wavelength    (1-D dataset, float64)
    /Spectrum/Jout          (1-D dataset, float32)
    /Spectrum/Jin           (1-D dataset, float32)
    /Spectrum/@nphotons     (attribute)
    /Spectrum/@taumax       (attribute)
    /Spectrum/@Nsc_gas      (attribute)
    ...

run_obs.h5
  /Scattered                                      (group, image-style)
```

```
    /Scattered/data         (3-D dataset, chunked + gzip)
    /Scattered/@CD1_1       (WCS attribute)
    ...
  /Direct
    /Direct/data
    /Direct/@EXTNAME        = 'Direct'

run_tau.h5
  /TAU_gas/data, /N_gas/data, [/TAU_dust/data when DGR > 0]
```

Sections appear in their write order on both the root group and inside each subgroup (creation-order tracking is enabled), so `h5py` iteration returns columns and HDUs in the same order LaRT wrote them. Datasets larger than 4096 elements are chunked and compressed with `gzip(4)`; smaller ones are stored contiguous. The on-disk storage type follows `par%out_bitpix` (auto / float32 / float64) just as for FITS.

11.3 Conversion utility

The Python helper `python/lart_io.py` reads either format and can convert between them losslessly:

```
python lart_io.py info    run.h5
python lart_io.py convert run.fits.gz run.h5
python lart_io.py convert run.h5      run.fits.gz
```

The round-trip preserves all data bit-identically; only string keyword names are uppercased on the FITS leg, matching the FITS card-name convention.

11.4 HDF5 library requirements

LaRT links against the HDF5 Fortran library (`libhdf5_fortran.a` plus the C core `libhdf5.a`, version ≥ 1.14). The Fortran `.mod` files are compiler-specific: an HDF5 build linked against gfortran cannot be used with Intel `ifx` or `ifort`, and vice versa. Set `par%HDF5_PREFIX` on the `make` command line to choose a build that matches the LaRT compiler.

12. Dust and Polarization

Parameter	Default	Description
`par%DGR`	0.0	Dust-to-gas ratio relative to the Milky Way value
`par%albedo`	0.32536	Single-scattering albedo ω
`par%hgg`	0.67617	Henyey–Greenstein asymmetry parameter g
`par%use_stokes`	`.false.`	Enable full Stokes polarization RT
`par%scatt_mat_file`	`''`	Mueller matrix file for polarized dust scattering
`par%use_reduced_wgt`	`.false.`	Reduce photon weight by albedo instead of destroying absorbed photons

When `par%DGR` = 0 (default), there is no dust and all photon weight is conserved. Non-zero `par%DGR` adds dust extinction and Henyey–Greenstein scattering. When `par%use_stokes` = `.true.`, a Mueller matrix file must be provided; files for the Weingartner–Draine Milky Way model are included in the `data/` directory.

13. Parallelism

Parameter	Default	Description
`par%no_photons`	10^6	Total photon packets (floating-point, converted to int64)
`par%use_master_slave`	`.true.`	Dynamic load balancing via master–worker algorithm
`par%num_send_at_once`	100	Photons dispatched to each worker per batch
`par%nprint`	10^5	Progress-report interval [photons]
`par%iseed`	auto	Random seed; defaults derived from MPI rank and system clock

With `par%use_master_slave = .true.`, one MPI rank acts as dispatcher and the rest as workers. This balances workloads when scattering rates vary across photon packets. With `.false.`, each rank processes an equal share of `par%no_photons`.

14. Complete Parameter Reference

Table 1 lists commonly used parameters. Parameters marked † are new or changed in v2.10.

Table 1: LaRT parameter reference.

Parameter	**Default**	**Description**
Simulation control		
`par%no_photons`	10^6	Number of photon packets
`par%nprint`	10^5	Progress interval [photons]
`par%iseed`	auto	Random seed
Cartesian grid		
`par%nx`, `par%ny`, `par%nz`	101	Cells per axis
`par%xmax`, `par%ymax`, `par%zmax`	1.0	Half-box size (code units)
`par%rmax`	–999	Spherical boundary radius
`par%rmin`	–999	Inner cavity radius
`par%geometry`	`'sphere'`	J output geometry (§4.2)
`par%xyz_symmetry`	`.false.`	$+x,+y,+z$ octant only
`par%xy_symmetry`	`.false.`	$+x,+y$ quadrant only
`par%xy_periodic`	`.false.`	Periodic in x,y

continued on next page

(continued)

Parameter	Default	Description
par%distance_unit	''	Physical unit ('kpc', 'pc', 'au')
par%distance2cm	1.0	1 code unit in cm
Grid mode		
par%grid_type†	'car'	v2.10+: 'car', 'amr', 'clump'
par%use_amr_grid	.false.	v2.00 AMR flag
par%use_clump_medium	.false.	v2.00 clump flag
Source		
par%source_geometry	'point'	See §5.1
par%xs_point, par%ys_point, par%zs_point	0.0	Point source position
par%source_rmax	–999	Source truncation radius
par%source_rscale	–999	Radial scale length
par%source_zscale	0.0	Vertical scale height
par%sersic_m	4.0	Sérsic index
par%Reff	1.0	Effective radius
par%comoving_source	.true.	Source co-moving with local gas
par%emiss_file	''	Emissivity FITS cube
par%star_file	''	Point-star list (x, y, z, L)
Line and spectrum		
par%line_id	Lyα	Atomic transition (§5.3)
par%spectral_type	'monochromatic'	Emission frequency distribution
par%temperature	10^4 K	Gas temperature
par%temperature0	$= T$	Source temperature for voigt0
par%bturb	0.0	Turbulent dispersion [km s^{-1}]
par%gaussian_sigma_vel	12.84	Gaussian σ [km s^{-1}] for spectral_type='gaussian'
par%gaussian_FWHM_vel	–999	Gaussian FWHM [km s^{-1}]; overrides σ if > 0
par%EW_line	0.0	Equivalent width [Å] for 'continuum+gaussian'
par%continuum_normalize	.true.	Normalize output so continuum level = 1

continued on next page

(continued)

Parameter	Default	Description
`par%xfreq0`	0.0	Frequency offset for emitted photons
`par%line_prof_file`	''	Custom line profile file
`par%line_prof_file_type`	0	0: (x, I); 1: (λ, I)
Frequency grid		
`par%nxfreq`	121	Spectral bins
`par%xfreq_min`, `par%xfreq_max`	auto	x-grid limits
`par%nwavelength`	0	Wavelength bins (overrides x-grid)
`par%wavelength_min`, `par%wavelength_max`	—	[Å]
`par%nvelocity`	0	Velocity bins (overrides x-grid)
`par%velocity_min`, `par%velocity_max`	—	[$\mathrm{km\,s^{-1}}$]
Optical depth and density		
`par%taumax`	—	Line-center τ_0: center $\rightarrow +z$ boundary
`par%tauhomo`	—	Same, uniform-density equivalent
`par%N_HImax`	—	Column density: center $\rightarrow +z$ [$\mathrm{cm^{-2}}$]
`par%N_HIhomo`	—	Same, uniform-density equivalent
`par%N_gasmax`	—	Ion column density [$\mathrm{cm^{-2}}$] (metal lines with `ion_model='none'`)
`par%density_rscale`	–999	Exponential density scale radius
`par%density_alpha`	0.0	Power-law density index: $n(r) \propto (r_{\max}/r)^{\alpha_D}$
External data		
`par%dens_file`	''	Density FITS cube [$\mathrm{cm^{-3}}$]
`par%temp_file`	''	Temperature FITS cube [K]
`par%velo_file`	''	Velocity FITS cube [$\mathrm{km\,s^{-1}}$]
`par%input_field`	''	Base name for `.dens/.temp/.velo.fits.gz`
Velocity		
`par%velocity_type`	''	Velocity law (§6.4)

continued on next page

(continued)

Parameter	Default	Description
`par%Vexp`	0.0	Expansion/infall speed [$\mathrm{km\,s^{-1}}$]
`par%velocity_alpha`	1.0	Power-law velocity exponent for `'power_law'`
`par%Vx`, `par%Vy`, `par%Vz`	0.0	Uniform velocity [$\mathrm{km\,s^{-1}}$]
`par%Vpeak`	0.0	Peak velocity for SSH [$\mathrm{km\,s^{-1}}$]
`par%rpeak`	0.0	Radius at peak for SSH
`par%DeltaV`	0.0	Velocity increment for SSH [$\mathrm{km\,s^{-1}}$]
`par%Vrot`	0.0	Rotation speed [$\mathrm{km\,s^{-1}}$]
`par%recoil`	`.false.`	Photon recoil in scattering
AMR		
`par%amr_type`	`'ramses'`	`'generic'` or `'ramses'`
`par%amr_file`	`''`	AMR data path
`par%amr_snapnum`	1	RAMSES snapshot number
Clumpy medium		
`par%clump_radius`	—	Clump radius
`par%clump_tau0`	—	Line-center τ_0 per clump
`par%clump_f_cov`	—	Covering factor C_f
`par%clump_f_vol`	—	Volume filling factor f_V
`par%clump_N_clumps`	—	Number of clumps N
`par%clump_sigma_v`	0.0	Random velocity dispersion [$\mathrm{km\,s^{-1}}$]
`par%save_clump_info`	`.false.`	Write clump table to FITS
Observers and peel-off		
`par%nxim`, `par%nyim`	0	Image pixels; peel-off active when > 0
`par%dxim`, `par%dyim`	auto	Pixel size [deg]
`par%distance`	$100\,r_{\rm max}$	Observer–center distance
`par%obsx`, `par%obsy`, `par%obsz`	$(0,0,1)$	Observer direction
`par%alpha`, `par%beta`	—	Azimuth, elevation [deg]
`par%gamma`	0.0	Detector-plane rotation [deg]
`par%save_peeloff`	`.false.`	1-D peel-off spectra
`par%save_peeloff_2D`	`.false.`	Peel-off spectral images

continued on next page

(continued)

Parameter	Default	Description
par%save_peeloff_3D	.false.	Full 3-D peel-off cube
par%save_direc0	.false.	Peel-off image without $e^{-\tau}$ (transparent-medium reference)
Output		
par%file_format	'hdf5'	Output container: 'hdf5' or 'fits'
par%out_file	''	Output file name (extension follows par%file_format)
par%out_merge	.false.	Accumulate into existing file
par%save_backup	.false.	Keep _NN copy of prior output on merge
par%out_bitpix	0	Storage precision: 0 = auto, –32, –64
par%save_Jin	.false.	Save injected spectrum
par%save_Jabs	.false.	Save absorbed spectrum
par%save_Jmu	.false.	Save angular spectrum $J(x;\mu)$
par%nmu	11	Number of μ bins for Jmu
par%save_all	.false.	Save full 3-D $J(\nu,x,y,z)$
par%save_radial_profile	.false.	Save 1-D radial J
par%save_sightline_tau	.false.	Save sightline τ
par%intensity_unit	''	Output header unit label
Dust		
par%DGR	0.0	Dust-to-gas ratio
par%albedo	0.32536	Single-scattering albedo
par%hgg	0.67617	Henyey–Greenstein asymmetry g
par%use_stokes	.false.	Polarization RT
par%scatt_mat_file	''	Mueller matrix file
par%use_reduced_wgt	.false.	Reduce weight instead of destroy
Parallelism		
par%use_master_slave	.true.	Dynamic load balancing
par%num_send_at_once	100	Photons per worker batch

15. Python Utilities

A small set of post-processing scripts is shipped under `python/` in each version's source tree. They depend on `numpy`, `astropy` (for FITS inputs) and/or `h5py` (for HDF5 inputs), and (for plotting scripts) `matplotlib`. All scripts read both formats transparently via the `lart_io.py` backend (§15.2).

15.1 Loading and analysing output: `read_lart.py`

`read_lart.py` loads a LaRT spectral output (FITS or HDF5) into a rich `LaRTOutput` dataclass that aggregates the angle-integrated spectrum, the optional $J(\mu, x)$ output, and every peel-off observer file alongside it. The dataclass exposes a small library of plotting and analysis methods so common diagnostics can be produced in a few lines. The script accepts the input-file name in any of four forms:

- full input file: `t1tau2.in`
- stem only: `t1tau2` (`.in` is added automatically)
- the HDF5 file: `t1tau2.h5` (also `.hdf5`)
- the FITS file: `t1tau2.fits.gz` (also `.fits`)

When given an input file, the reader honors `par%out_file` from the namelist first, then searches for any LaRT output stem under all recognised extensions (HDF5 preferred) via `lart_io.find_lart_file`. If the output file is still absent but the namelist sets `par%clump_input_file` pointing at an existing clump file, `read_lart` prints a one-line notice and returns a "clumps-only" `LaRTOutput` with the spectrum arrays unset (`out.is_clumps_only == True`) but with the clump population already loaded into `out.clumps` (a `ClumpsOutput`, see below). `out.plot_clump_slice()` therefore works on the pre-run dataclass, so the clump population can be inspected before launching the radiative-transfer run. Peel-off observer files are auto-discovered the same way — both the single-observer (`<stem>_obs.h5`) and the multi-observer (`<stem>_obs_000.h5`, `_obs_001.h5`, ...) layouts are supported, in either format.

Module API.

```
from read_lart import read_lart
out = read_lart('t1tau2.in')      # FITS or HDF5 -- format-agnostic

# 1-D arrays, length nxfreq
out.xfreq, out.velocity, out.wavelength
out.Jout                    # always present
out.Jin, out.Jabs, out.Jabs2     # optional -- None if not in file

# Jmu output -- present only if par%save_Jmu = .true. was used
out.mu          # bin centers, length nmu
out.mu_edges    # bin edges, length nmu+1
out.Jmu         # shape (nmu, nxfreq); None if absent
out.nmu, out.mu_min, out.dmu
```

```

# Per-observer peel-off data (one PeelObservation per observer file)
out.peelings    # list, sorted by (beta, alpha, gamma)

# Raw section attribute dicts (case-insensitive; legacy uppercase
# keys like spec_hdr['EXTNAME'] also work)
out.spectrum_header, out.jmu_header
```

Every optional array is `None` when its section is not present, so the reader works on outputs from any LaRT version and either format. The `spectrum_header` and `jmu_header` dicts are filled with both the original-case keys (as they appear in the underlying file) and uppercase aliases, so legacy callers using FITS-style key access (e.g. `spec_hdr['EXTNAME']`) keep working on HDF5 outputs.

Peel-off observer accessor. Each entry in `out.peelings` is a `PeelObservation` dataclass with fields `alpha`, `beta`, `gamma`, `distance`, `nphotons`, `nxim`, `nyim`, `sr_pix`, the scattered + direct cubes `scatt`, `direc`, `direc0` (each shape `(nyim, nxim, nxfreq)`), and the combined `cube = scatt + direc`. Two helper methods are provided:

```
po = out.peelings[0]
spec = po.average_spectrum()        # area-mean spectrum, length nxfreq
v0, vsig = po.velocity_moment_map(weight='cube')   # (nyim, nxim) each
```

Plotting Jmu. `LaRTOutput.plot_jmu()` draws the angular spectrum directly:

```
out = read_lart('t1tau3.in')
out.plot_jmu(kind='lines', x='velocity')   # one curve per mu bin
out.plot_jmu(kind='image', x='velocity')   # 2-D heatmap (x vs mu)
```

The `x=` keyword selects the abscissa (`'velocity'`, `'xfreq'`, or `'wavelength'`); the `kind=` keyword toggles between line ensemble (colored by μ) and a 2-D image. The line view also overplots `Jout` as a black dashed reference. Pass `ax=` to draw onto an existing axes, `savefig='out.png'` to save to disk. Axis limits accept either the tuple form `xlim=(lo,hi)` / `ylim=(lo,hi)` or the per-bound shorthand `xmin/xmax/ymin/ymax`; the per-bound override takes precedence when both are given:

```
out.plot_jmu(kind='lines', xlim=(-500, 500), ymin=0)
out.plot_jmu(kind='lines', xmax=300)         # left edge auto-scaled
```

Peel-off analysis plots. Several methods operate on the auto-discovered `out.peelings` list and produce moment maps, per-observer spectra, and radial profiles:

```
out.plot_peeling_map(weight='cube',     mode='intensity')
out.plot_peeling_map(weight='velocity', mode='v_los')
out.plot_peeling_map(weight='velocity', mode='sigma_v')
out.plot_peeling_spectrum()              # one curve per observer
out.plot_peeling_radial_profile()        # azimuthally averaged
out.plot_velocity_moment_map(po, mode='v_los')   # single observer
```

The `plot_peel_jmu_compare()` method overlays each observer's peel-off spectrum on the matching $J(\mu,x)$ slice as an isotropy diagnostic, and `plot_clump_slice()` draws the 2D cross-section of the clumpy medium at a chosen plane (it opens the matching `<stem>_clumps.h5` / `.fits.gz` file automatically). All plotting methods accept `ax=`, `savefig=`, and `xlim`/`ylim` (or `xmin`/`xmax`/`ymin`/`ymax`) the same way as `plot_jmu`.

Command-line usage.

```
python read_lart.py t1tau2          # auto-resolves to t1tau2.in
python read_lart.py t1tau2.in
python read_lart.py t1tau2.h5       # HDF5 output
python read_lart.py t1tau2.fits.gz  # FITS output
```

prints a one-screen summary of which arrays are present and, when `Jmu` is in the file, the per-bin $\max_x J(x;\mu) / \max_x J_{\rm out}(x)$ ratio (a quick isotropy sanity check for homogeneous models).

Standalone clump access: `read_clumps` / `read_clump`. The clump population can also be loaded independently of any radiative- transfer output via the top-level helper:

```
from read_lart import read_clumps      # alias: read_clump
co = read_clumps('foo_clumps.h5')      # direct clump file
co = read_clumps('run.in')             # via par%clump_input_file
co = read_clumps('run')                # stem -> tries .in, then _clumps.*
print(co.summary())
co.plot_clump_slice(axis='z')
```

The returned `ClumpsOutput` dataclass exposes the per-clump arrays (`x, y, z, vx, vy, vz, radius, rhokap, density, temp`, plus the convenience attributes `pos`, `vel`, `n_clumps`, `sphere_r`, `f_vol`, `f_cov`, `distance_unit`, `distance2cm`), the case-insensitive attribute dict `attrs` (with `co.attr(name)` helper), and the `plot_clump_slice()` method. The opacity column in the file is auto-detected with priority `RHOKAP` → `DENSITY` → `DENS`: `RHOKAP` populates `co.rhokap` (opacity per code unit), while `DENSITY` / `DENS` populate `co.density` (n_{HI} [cm^{-3}]); `co.opacity_col` records which column was found. `co.distance_unit` / `co.distance2cm` expose the `DISTUNIT` / `DIST_CM` attributes from the file (LaRT's `par%distance_unit` / `par%distance2cm` at write time). When the `F_VOL` / `F_COV` attributes are missing from the file, the `f_vol` / `f_cov` properties fall back to `compute_f_vol()` / `compute_f_cov()`, which evaluate the same formulas as LaRT's `write_clumps_fits()` from-file branch in `clump_mod.f90`: $f_{\rm vol} = \sum r_i^3/(R^3 - r_{\rm min}^3)$ and $f_{\rm cov} = \frac{3}{4}\sum r_i^2/(R^2 + R\,r_{\rm min} + r_{\rm min}^2)$, with R taken from `SPHERE_R` (or `RMAX`) and $r_{\rm min}$ from `RMIN` (default 0). The two `compute_*` methods can also be called explicitly to recompute regardless of the stored attribute. Inside `LaRTOutput` the same dataclass lives at `out.clumps` (lazy- loaded on the first `plot_clump_slice` call, or populated eagerly in the clumps-only pre-run state described above).

Other scripts. The following plotting helpers are also shipped:

- `plot_spectrum.py`, `plot_map.py`, `plot_intensity_profile.py`, `plot_Pa.py`: standard quick-look plots.
- `check_flux.py`, `flux_check.py`: flux-conservation diagnostics.

- AMR_grid/: helpers for generating generic AMR data files (see AMR_grid/make_amr_grid.py, make_amr_sphere_radial.py).

15.2 Format-agnostic reader and FITS $\leftrightarrow$ HDF5 converter: lart_io.py

python/lart_io.py provides a single load_lart(path) entry point that reads either a FITS or an HDF5 LaRT output and returns a uniform LartFile structure (one Section per HDU / group, with .data or .columns plus .attrs). It also converts between the two formats by extension. Requires numpy plus astropy (for FITS) and/or h5py (for HDF5); each backend is only imported when needed.

Module API.

```
from lart_io import load_lart, convert

f = load_lart('run.h5')            # works on .fits.gz too
spec = f['Spectrum']               # a Section
jout = spec.columns['Jout']        # 1-D numpy array
nph  = spec.attrs['nphotons']

scat = load_lart('run_obs.h5')['Scattered']
cube = scat.data                   # 3-D array (nxim, nyim, nxfreq)

convert('run.fits.gz', 'run.h5')  # both directions by extension
convert('run.h5',      'run.fits.gz')
```

The round-trip preserves all numerical data bit-identically; string keyword names are normalized to uppercase on the FITS leg (FITS card-name convention), which is the only visible difference between the two forms.

Command-line usage.

```
python lart_io.py info   run.h5
python lart_io.py info   run.fits.gz
python lart_io.py convert  run.fits.gz  run.h5
python lart_io.py convert  run.h5       run.fits.gz
```

16. Example Input Files

16.1 Uniform Sphere, Lyα, Static

```
&parameters
 par%no_photons      = 1e6
 par%temperature     = 1.0e4
 par%taumax          = 1.0e4
 par%rmax            = 1.0
```

```
 par%nx = 101, par%ny = 101, par%nz = 101
 par%source_geometry = 'point'
 par%spectral_type  = 'voigt'
 par%nxfreq         = 121
/
```

16.2 Sphere with Hubble Outflow and Peel-off

```
&parameters
 par%no_photons     = 1e6
 par%temperature    = 1.0e4
 par%taumax         = 1.0e4
 par%rmax           = 1.0
 par%nx = 101, par%ny = 101, par%nz = 101
 par%source_geometry = 'point'
 par%spectral_type  = 'voigt'
 par%velocity_type  = 'hubble'
 par%Vexp           = 200.0          ! km/s outflow
 par%nxfreq         = 121
 par%nxim = 129, par%nyim = 129
 par%alpha = 0.0  45.0  90.0
 par%beta  = 0.0   0.0   0.0
 par%save_peeloff_2D = .true.
/
```

16.3 Realistic Galaxy (FITS Density Field, Stokes Polarization)

```
&parameters
 par%no_photons    = 1e8
 par%dens_file     = 'galaxy.dens.fits.gz'
 par%temp_file     = 'galaxy.temp.fits.gz'
 par%velo_file     = 'galaxy.velo.fits.gz'
 par%taumax        = 1.0e4           ! rescale to this optical depth
 par%distance_unit = 'kpc'
 par%source_geometry = 'exponential'
 par%source_rscale = 3.0
 par%source_zscale = 0.5
 par%spectral_type = 'voigt'
 par%DGR           = 1.0
 par%use_stokes    = .true.
```

```
 par%nxfreq        = 121
 par%nxim = 200, par%nyim = 200
 par%save_peeloff_2D = .true.
/
```

16.4 AMR Sphere (Generic Text Format)

```
&parameters
 par%use_amr_grid   = .true.          ! or par%grid_type = 'amr' in v2.10
 par%amr_type       = 'generic'
 par%amr_file       = 'sphere_amr.dat'
 par%no_photons     = 1e6
 par%taumax         = 1.0e4
 par%source_geometry = 'point'
 par%spectral_type  = 'voigt'
 par%nxfreq         = 121
 par%nxim = 100, par%nyim = 100
 par%save_peeloff_2D = .true.
 par%save_Jin       = .true.
/
```

16.5 Clumpy Sphere with Random Velocities

```
&parameters
 par%use_clump_medium = .true.       ! or par%grid_type = 'clump' in v2.10
 par%rmax             = 1.0
 par%clump_radius     = 0.01
 par%clump_f_cov      = 5.0
 par%clump_tau0       = 1.0e3
 par%temperature      = 1.0e4
 par%clump_sigma_v    = 50.0         ! km/s random dispersion
 par%no_photons       = 1e6
 par%spectral_type    = 'voigt'
 par%nxfreq           = 301
 par%velocity_min = -200.0
 par%velocity_max =  200.0
 par%save_clump_info  = .true.
/
```